

CollabDocs - Collaborative...

PAST DUE

Past due

Core project

Group work

0 / 30

Project brief Your submission Your group

Brief

1. Project Overview

Build the backend API for **CollabDocs** - a platform where users can create workspaces, invite collaborators, write and version documents, leave comments, and control access with role-based permissions. Think of it as a simplified Notion or Google Docs, API-only.

Your group will design the data models, implement all API endpoints using Django REST Framework, handle validation and error responses, write a custom middleware, use transactions to ensure data integrity, and demonstrate query optimisation. The frontend is out of scope; Postman is your client.

What this assignment covers

Python OOP, Django fundamentals, DRF ViewSets and Serializers, PostgreSQL models and migrations, QuerySets, filtering, aggregations, transactions, middleware, and signals. Every requirement maps to something taught live in class. Your job is to own it end-to-end.

2. Data Models

Your application must implement the following **8 models**. All primary keys must be UUIDField with auto-generation. You may add fields beyond the minimum listed.

2.1 Model Specifications

User		
Field	Type	Notes
id	UUIDField (PK)	uuid.uuid4, editable=False
first_name	CharField(50)	
last_name	CharField(50)	
email	CharField(254)	unique=True — use CharField, not EmailField
phone	CharField(15)	unique=True
created_at	DateTimeField	auto_now_add=True

Ask me anything

Workspace		
Field	Type	Notes
id	UUIDField (PK)	uuid.uuid4, editable=False
name	CharField(255)	
owner	FK → User	on_delete=CASCADE. Owner must also be added as WorkspaceMember with role=admin
is_active	BooleanField	default=True
created_at	DateTimeField	auto_now_add=True

WorkspaceMember		
Field	Type	Notes
id	UUIDField (PK)	uuid.uuid4, editable=False
workspace	FK → Workspace	on_delete=CASCADE
user	FK → User	on_delete=CASCADE
role	CharField	TextChoices: admin, editor, viewer
joined_at	DateTimeField	auto_now_add=True

Add a UniqueConstraint in the model's Meta class on (workspace, user). Pattern: class Meta: constraints = [models.UniqueConstraint(fields=['workspace','user'], name='unique_workspace_member')]

Document		
Field	Type	Notes
id	UUIDField (PK)	uuid.uuid4, editable=False
title	CharField(255)	
content	TextField	
workspace	FK → Workspace	on_delete=CASCADE
created_by	FK → User	on_delete=SET_NULL, null=True
status	CharField	TextChoices: draft, published, archived
updated_at	DateTimeField	auto_now=True

⚠ Every save must create a new DocumentVersion inside the same transaction.atomic() block.

DocumentVersion		
Field	Type	Notes
id	UUIDField (PK)	uuid.uuid4, editable=False
document	FK → Document	on_delete=CASCADE
content	TextField	Snapshot of content at save time
version_number	PositiveIntegerField	Auto-increment per document — see implementation note below
saved_by	FK → User	on_delete=SET_NULL, null=True
saved_at	DateTimeField	auto_now_add=True

⚠ version_number: inside your create/update logic, compute it as Document.versions.count() + 1 before saving, this gives per-document numbering without a global counter.

Ask me anything

Comment		
Field	Type	Notes
id	UUIDField (PK)	uuid.uuid4, editable=False
document	FK → Document	on_delete=CASCADE
author	FK → User	on_delete=SET_NULL, null=True
content	TextField	
parent	FK → self	null=True, blank=True, on_delete=SET_NULL — enables threaded replies
created_at	DateTimeField	auto_now_add=True

⚠ *Self-referential FK: `parent = models.ForeignKey('self', null=True, blank=True, on_delete=models.SET_NULL, related_name='replies')`. A comment with no parent is a top-level comment.*

Tag		
Field	Type	Notes
id	UUIDField (PK)	uuid.uuid4, editable=False
name	CharField(100)	unique=True
documents	ManyToManyField	ManyToManyField(Document, related_name='tags', blank=True) — declare on Tag, not Document

⚠ *ManyToManyField: Django creates a join table automatically. To add tags to a document use `tag.documents.add(doc)` or `doc.tags.add(tag)`. To filter docs by tag: `Document.objects.filter(tags__name='python')`.*

AuditLog		
Field	Type	Notes
id	UUIDField (PK)	uuid.uuid4, editable=False
actor	FK → User	on_delete=SET_NULL, null=True
action	CharField(50)	e.g. 'created', 'updated'
model_name	CharField(100)	e.g. 'Document'
object_id	CharField(100)	UUID as string
timestamp	DateTimeField	auto_now_add=True

⚠ *Written automatically via a `post_save` signal on Document. Use `instance._state.adding` to distinguish created vs updated.*

2.2 Constraints to implement

- WorkspaceMember: UniqueConstraint on (workspace, user) in Meta class
- DocumentVersion: `version_number` computed as `Document.versions.count() + 1` inside the atomic block
- Workspace creation: owner added as WorkspaceMember with `role=admin` inside a `single.transaction.atomic()`
- [Workspace.is_active](#): BooleanField with `default=True`
- Document.status and WorkspaceMember.role: TextChoices

3. Required API Endpoints

Use `ModelViewSet` for standard CRUD and `@action` decorators for custom endpoints. All endpoints must return the correct HTTP status code and a meaningful error message.

Users

Ask me anything

Method	Endpoint	What it does	Concept tested
POST	/api/users/	Create a user	ModelViewSet, ModelSerializer, validation
GET	/api/users/{id}/	Get user by ID	UUIDField PK, serializer response

Workspaces

Method	Endpoint	What it does	Concept tested
POST	/api/workspaces/	Create workspace + auto-add owner as admin member	transaction.atomic(), override create()
GET	/api/workspaces/{id}/	Get workspace with member count	annotate(), Count
POST	/api/workspaces/{id}/members/	Add a member with a role	FK, UniqueConstraint handling, custom validation
GET	/api/workspaces/{id}/members/	List all members with their roles	select_related, serializer nesting
GET	/api/workspaces/{id}/summary/	Doc count, member count, total comments	@action, aggregate(), annotate(), Count



Documents

Method	Endpoint	What it does	Concept tested
POST	/api/documents/	Create document + first version (atomic)	transaction.atomic(), version_number logic
PUT	/api/documents/{id}/	Update document content — saves a new version	transaction.atomic(), override update()
GET	/api/documents/	List docs — filter by workspace, status, tag name; search title	Q objects, query params, filter(), __icontains
GET	/api/documents/{id}/versions/	All versions of a document in order	filter(), order_by(), FK reverse lookup
GET	/api/documents/{id}/stats/	Version count, comment count, contributor count	@action, aggregate(), annotate(), Count
POST	/api/documents/{id}/tags/	Add one or more tags to a document	@action, ManyToManyField .add()

Comments

Method	Endpoint	What it does	Concept tested
POST	/api/comments/	Add a top-level comment or a reply	Self-referential FK, SerializerMethodField, validation
GET	/api/comments/?document={id}	List all comments for a document (threaded)	filter(), select_related, query params

Tags & Audit Logs

Method	Endpoint	What it does	Concept tested
POST	/api/tags/	Create a tag	ModelSerializer, unique constraint handling
GET	/api/audit-logs/	Audit logs filtered by actor ID and date range	filter(), __gte, __lte, query params

Ask me anything

3.1 Middleware requirement

Implement a custom request logging middleware that prints the following for every request:

- HTTP method (GET, POST, PUT, DELETE)
- Endpoint path
- Response status code
- Time taken in milliseconds

Register it in [settings.py](#) MIDDLEWARE list. A Django middleware is a class with `init(self, get_response)` and `call(self, request)` methods. Record the time before calling `get_response(request)` and after, then print the difference.

4. Technical Requirements

4.1 Models and database

- All PKs: UUIDField with `uuid.uuid4` and `editable=False`
- Use TextChoices for all enums - no raw string values
- UniqueConstraint on WorkspaceMember (workspace, user) in the model's Meta class
- ManyToManyField between Tag and Document
- Self-referential ForeignKey on Comment for threaded replies
- All migrations committed and applying cleanly from scratch

4.2 Serializers and views

- Use ModelViewSet wherever CRUD is standard
- Use @action decorators for all non-standard endpoints (stats, summary, tags, versions)
- Implement at least two SerializerMethodField uses across the project
- Write custom validation in at least two serializers
- All error responses: meaningful message, correct HTTP code (400, 404, 409 where appropriate)
- Do not use raw APIView where ModelViewSet is appropriate

4.3 QuerySets and filtering

- Use select_related on every endpoint returning nested user, workspace, or document data
- Use filter() with lookups (gte, lte, in, __icontains) on all list endpoints that support filtering
- Use Q objects for OR filtering on the document list endpoint
- Use aggregate() or annotate() with Count in at least 3 endpoints
- Use values_list() where only IDs are needed

4.4 Transactions and data integrity

- Wrap workspace creation + member add in a single transaction.atomic()
- Wrap document save + version creation in a single transaction.atomic()
- AuditLog must be written inside the same atomic block as the action it records
- Handle DoesNotExist and IntegrityError explicitly, catch IntegrityError on WorkspaceMember creation to return a 409 when a duplicate member is added

4.5 Middleware and signals

- Custom request logging middleware as described and registered in [settings.py](#)

Ask me anything

- post_save signal on Document, write it in [signals.py](#), connect it in AppConfig.ready(). Use instance._state.adding to detect create vs update. Write an AuditLog entry for each
- The signal must record: actor (created_by), action ('created' or 'updated'), model_name ('Document'), object_id

4.6 Project setup

- PostgreSQL via .env, never hardcoded credentials
- requirements.txt with all dependencies pinned
- [README.md](#) with setup, how to run the server, how to apply migrations
- Postman collection (.json) committed to the repository root
- All 17 endpoints tested and working in Postman before submission.

5. Marking Rubric

Area	What we look for	Marks
Models & migrations	Correct field types, UUID PKs, TextChoices, UniqueConstraint in Meta, ManyToManyField, self-referential FK, clean migrations	20
Serializers & validation	ModelSerializer, two SerializerMethodField uses, custom validators, meaningful error messages, correct status codes	15
ViewSet & routing	ModelViewSet for standard CRUD, @action for custom endpoints, override create() and update(), DefaultRouter	15
QuerySets & filtering	select_related on all nested data, filter() with __ lookups incl. __icontains, Q objects, values_list(), query params	15
Aggregations	aggregate() and annotate() with Count used correctly in at least 3 endpoints (stats, summary, workspace detail)	10
Transactions & integrity	transaction.atomic() for workspace + document flows; AuditLog in same block; DoesNotExist and IntegrityError handled	10
Middleware & signals	Working request logging middleware; post_save signal writing AuditLog; both registered correctly in settings.py	10
Code quality & setup	README, .env.example, requirements.txt, clean project structure, readable code	5
TOTAL		100



Submission guidelines

Submission Requirements

⚠️ All three items are required

An incomplete submission will be marked down regardless of the quality of the code.

6.1 GitHub repository (required)

- Public repository with the full source code
- .env.example showing all required environment variables - do not commit the real .env
- requirements.txt with all dependencies pinned
- All migrations committed and applying cleanly
- [README.md](#) with setup instructions, how to run the server, and how to apply migrations
- Postman collection .json committed to the repository root

6.2 Postman collection (required)

- Exported .json covering all 17 endpoints
- Sample request bodies for all POST and PUT endpoints
- Organised into folders: Users, Workspaces, Documents, Comments, Tags, Audit Logs

Ask me anything

6.3 Demo video (required, 5–10 minutes)

- Screen recorded with audio - walk through the app via Postman
- Must show: one complete atomic transaction with rollback on failure
- Must show: middleware logs appearing in the console during requests
- Must show: at least one aggregation endpoint (stats or summary)
- Must show: AuditLog being written by the signal after a document update
- Upload to Loom or Google Drive - include the link in your README

Deliverables



Video



GitHub link



Hey there, I'm your guide for this project!

Ask me anything regarding this brief, I'd be happy to help you out.

Design data models

Implement API endpoints

Write custom middleware

Ensure data integrity



Ask doubts or share resources

Add comment

Share tips, resources or anything else related to the assignment here

B *I* u

Post

The comments can be seen by everyone in the cohort

Comments: (0)

No comments yet. Be the first one to start a discussion.

Ask me anything